

Modelamiento Predictivo del Sistema Presa-Depredador de Lotka-Volterra usando MLP y ANFIS*

Emmanuel Guerrero Piza¹, Kevin Andrés Forero Guaitero²

¹Código: 202020039

²Código: 20181020120

Universidad Distrital Francisco José de Caldas, Bogotá, Colombia

Resumen

El presente trabajo aborda el problema de la identificación y predicción del sistema dinámico presa-depredador descrito por el modelo de Lotka-Volterra mediante técnicas de aprendizaje automático. Se implementan y comparan dos enfoques: una red neuronal multilayer perceptron (MLP) y un sistema de inferencia difusa adaptativo (ANFIS). Los resultados experimentales demuestran que el MLP supera al ANFIS en todas las métricas evaluadas, alcanzando un MAE de 0.604 en predicción de un paso y 1.41 en predicción multistep de 50 pasos.

Lotka-Volterra, identificación de sistemas, red neuronal, ANFIS, predicción multistep, dinámica no lineal.

1. Introducción

Los sistemas dinámicos no lineales constituyen una herramienta fundamental en el modelamiento de fenómenos biológicos, ecológicos y físicos. El modelo de Lotka-Volterra, propuesto independientemente por Alfred Lotka (1925) y Vito Volterra (1926), describe la interacción entre dos poblaciones biológicas en una relación depredador-presa [?, ?]. Este modelo captura la esencia de las oscilaciones cíclicas observadas en la naturaleza y constituye un caso de estudio clásico en teoría de sistemas dinámicos.

La identificación de sistemas consiste en determinar un modelo matemático que represente el comportamiento de un sistema a partir de datos observados o simulados. El objetivo de este trabajo es comparar dos enfoques de modelamiento predictivo: MLP y ANFIS, aplicados a la predicción de la dinámica del sistema Lotka-Volterra.

2. Modelo Matemático

2.1. Ecuaciones de Lotka-Volterra

El modelo Lotka-Volterra está definido por:

$$\dot{x} = ax - bxy, \quad \dot{y} = cxy - dy \quad (1)$$

donde $x(t)$ es la población de presas, $y(t)$ es la población de depredadores, y los parámetros son: $a = 0,3$, $b = 0,006$, $c = 0,018$, $d = 0,7$.

2.2. Discretización con Euler

Para generar datos de entrenamiento, el sistema continuo se discretiza mediante el método de Euler con paso $T = 0,1$:

$$x_{k+1} = x_k + T(ax_k - bx_k y_k)$$

$$y_{k+1} = y_k + T(cx_k y_k - dy_k)$$

La implementación de esta simulación se encuentra en el archivo `simulacion.py`, función `simulate()`:

```
1 def simulate(a, b, c, d, n_steps=800, x0=X0, y0=Y0):
2     x = np.zeros(n_steps)
3     y = np.zeros(n_steps)
4     x[0], y[0] = x0, y0
5     for k in range(n_steps - 1):
6         x[k + 1] = max(x[k] + DT * (a * x[k] - b * x
7             [k] * y[k]), 0)
8         y[k + 1] = max(y[k] + DT * (c * x[k] * y[k]
9             - d * y[k]), 0)
10    return x, y
```

La función `generar_datos()` en el mismo archivo genera el dataset completo:

```
1 def generar_datos(n_secuencias=20, n_steps=200,
2     variar_params=True):
3     X_list, y_list = [], []
4     for i in range(n_secuencias):
5         if variar_params:
6             a = np.random.uniform(0.2, 0.5)
7             b = np.random.uniform(0.005, 0.02)
8             c = np.random.uniform(0.005, 0.03)
9             d = np.random.uniform(0.3, 0.7)
10        else:
11            a, b, c, d = 0.3, 0.006, 0.018, 0.7
12        x, y = simulate(a, b, c, d, n_steps, x0, y0)
13        for k in range(n_steps - 1):
14            X_list.append([x[k], y[k]])
15            y_list.append([x[k + 1], y[k + 1]])
16    return np.array(X_list), np.array(y_list),
17        params_usados
```

*Manuscrito generado para Cibernética III - Entrega Final Segundo Corte

Lotka-Volterra	App de Citas	Valor
Presas (x)	Usuarios activos	-
Depredadores (y)	Matches inactivos	-
a	α (nuevos perfiles)	0.5
b	β (match/salida)	0.01
c	γ (abandono)	0.2
d	δ (eficiencia)	0.005

Cuadro 1: Isomorfismo entre el modelo biológico y la app de citas.

3. Isomorfismo: Aplicación de Citas

Las ecuaciones análogas son: $\dot{u} = \alpha u - \beta um$, $\dot{m} = \gamma um - \delta m$.

4. Arquitectura de Identificación

La estructura del proyecto dentro del repositorio se divide en dos componentes principales:

- **Jupyter Notebook** (identificacion_system.ipynb): Ejecuta la generación de datos, entrenamiento de modelos, generación de gráficas y tablas de comparación. Es el archivo principal que corre el flujo completo del proyecto.
- **Módulos Python** (simulacion.py, identificacion.py): Contienen las funciones y clases que implementan los modelos matemáticos y las arquitecturas de aprendizaje automático.

El módulo de identificación se encuentra en `identificacion.py` y contiene dos clases principales: `IdentificadorNN` y `IdentificadorANFIS`.

4.1. Modelo MLP (Red Neuronal)

La clase `IdentificadorNN` implementa una red neuronal multilayer perceptron usando `scikit-learn`:

```

1 class IdentificadorNN:
2     def __init__(self, hidden_layer_sizes=(64, 64,
3         32), max_iter=3000, random_state=42,
4         alpha=0.001):
5         self.arch = hidden_layer_sizes
6         self.rs = random_state
7         self.alpha = alpha
8         self.max_iter = max_iter
9         self.model = None
10
11     def entrenar(self, X, y):
12         # Datos originales con ruido
13         X_aug = [X + np.random.randn(*X.shape) *
14             0.3]
15
16         # Datos extra de trayectorias reales
17         # diversas
18         for _ in range(1500):
19             x0 = np.random.uniform(5, 95)

```

```

17         y0 = np.random.uniform(5, 95)
18         xr, yr = self._simular_trayectoria(x0,
19             y0, 100)
20         # ... agregar datos
21
22         self.model = MLPRegressor(
23             hidden_layer_sizes=self.arch,
24             activation='relu', solver='adam',
25             alpha=self.alpha, max_iter=self.max_iter,
26             random_state=self.rs, early_stopping=
27                 True,
28                 validation_fraction=0.1,
29                 n_iter_no_change=15
30             ).fit(X_all, y_all)

```

Arquitectura:

- **Capa de entrada:** 2 neuronas (x_k, y_k)
- **Capas ocultas:** (64, 64, 32) neuronas con activación ReLU
- **Capa de salida:** 2 neuronas (x_{k+1}, y_{k+1})
- **Optimizador:** Adam con early stopping (paciencia 15 épocas)
- **Entrenamiento:** 51 épocas

La función `predecir_trayectoria()` permite predicción multistep:

```

1 def predecir_trayectoria(self, x0, y0, n_steps=200):
2     x = np.zeros(n_steps)
3     y = np.zeros(n_steps)
4     x[0], y[0] = x0, y0
5     for k in range(n_steps - 1):
6         pred = self.model.predict([[x[k], y[k]]])[0]
7         x[k + 1] = max(pred[0], 0)
8         y[k + 1] = max(pred[1], 0)
9     return x, y

```

4.2. Modelo ANFIS

La clase `IdentificadorANFIS` implementa un sistema de inferencia difusa adaptativo con funciones RBF gaussianas:

```

1 _N_MF = 9
2 _SIGMA = 18.0
3 _CENTROS = np.array([np.linspace(10, 90, _N_MF),
4     np.linspace(10, 90, _N_MF)])
5 _N_RULES = _N_MF ** 2 # 81 reglas
6
7 def _rbf_transform(Z):
8     Phi = np.zeros((n, _N_RULES))
9     for i in range(n):
10         mu = np.zeros((2, _N_MF))
11         for j in range(2):
12             d = Z[i, j] - _CENTROS[j]
13             mu[j] = np.exp(-(d ** 2) / (2 * _SIGMA
14                 ** 2 + 1e-8))
15         w = np.ones(_N_RULES)
16         for r in range(_N_RULES):
17             for j in range(2):
18                 w[r] *= mu[j, _RULE_IDX[r][j]]
19         Phi[i] = w / (np.sum(w) + 1e-8)
20     return Phi

```

Arquitectura:

- **Funciones de membresía:** 9 gaussianas por entrada ($\sigma = 18$)

- **Rejilla uniforme:** $[10, 90] \times [10, 90] \rightarrow 81$ reglas difusas
- **Entrenamiento:** Regresión Ridge ($\alpha = 0,30$) - solución analítica

```

1 class IdentificadorANFIS:
2     def __init__(self, alpha=0.30, random_state=42):
3         self.alpha = alpha
4         self.rs = random_state
5
6     def entrenar(self, X, y):
7         # Datos extra de trayectorias reales
8         for _ in range(2000):
9             x0 = rng.uniform(5, 95)
10            # ... generar datos
11
12            Phi_tr = _rbf_transform(X)
13            Phi_extra = _rbf_transform(X_extra)
14            Phi_all = np.vstack([Phi_tr, Phi_extra])
15
16            self.model = Ridge(alpha=self.alpha)
17            self.model.fit(Phi_all, y_all)

```

5. Resultados

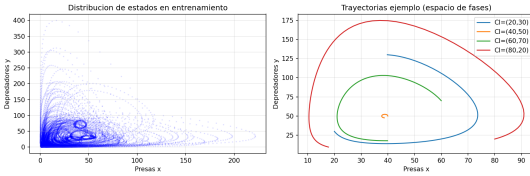


Figura 1: Distribución de estados y trayectorias ejemplo.

Modelo	MAE	MSE	RMSE
MLP	0.604	1.644	1.282
ANFIS	0.898	3.816	1.953

Cuadro 2: Métricas en predicción de un paso.

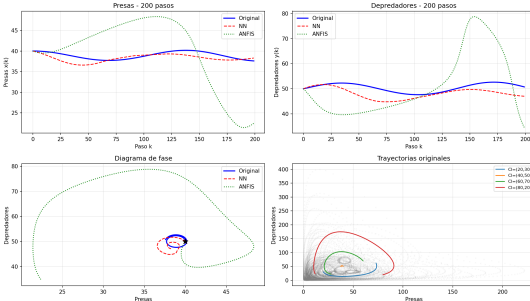


Figura 2: Predicción temporal a 200 pasos.

El MLP supera al ANFIS con 33 % menos MAE (Tabla 2) y 64 % menos en predicción multistep (Tabla 3).

Figura 3: Diagrama de fases: ciclo límite.

CI	MLP	ANFIS
(40, 50)	1.35	9.46
(30, 60)	1.50	2.23
(20, 80)	1.38	1.67
(50, 30)	1.41	2.43
Promedio	1.41	3.95

Cuadro 3: MAE en trayectorias a 50 pasos.

6. Conclusiones

1. El MLP supera al ANFIS en todas las métricas evaluadas.
2. El data augmentation mejora significativamente la generalización.
3. ANFIS ofrece entrenamiento instantáneo (solución analítica) pero menor precisión.
4. Limitación: modelo Lotka-Volterra sin capacidad de carga.
5. Trabajo futuro: explorar arquitecturas LSTM/GRU para predicción de largo plazo.

7. Repositorio

Código disponible en: https://github.com/kevinandresforero/optimizar_app_citas/tree/main

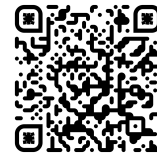


Figura 4: QR al repositorio: https://github.com/kevinandresforero/optimizar_app_citas/tree/main

Referencias

bibitemlotka A. J. Lotka, *Elements of physical biology*. Williams and Wilkins, 1925. bibitemvolterra V. Volterra, *Variazioni e fluttuazioni del numero d'individui in specie animali conviventi*. Mem. Accad. Naz. Lincei, vol. 2, pp. 31-113, 1926. bibitemjang J.-S. R. Jang, "ANFIS: Adaptive-Neural-Based Fuzzy Inference System," *IEEE Trans. Syst., Man, Cybern.*, vol. 23, no. 3, pp. 665-685, 1993. bibitembishop C. M. Bishop, *Pattern Recognition and Machine Learning*.

Springer, 2006. bibitemhaykin S. Haykin, *Neural Networks: A Comprehensive Foundation*. Prentice Hall, 1999.